

SLOVAK UNIVERSITY OF TECHNOLOGY IN BRATISLAVA

Faculty of Electrical Engineering and Information Technology

Registration number: FEI-184458-122378

**Translation codes constructions supported by
computer**

Bachelor's thesis

SLOVAK UNIVERSITY OF TECHNOLOGY IN BRATISLAVA
Faculty of Electrical Engineering and Information Technology

Registration number: FEI-184458-122378

**Translation codes constructions supported by
computer**

Bachelor's thesis

Study Programme:	Information and Communication Technologies
Study Field:	Computer Science
Training Workplace:	Institute of Multimedia Information and Communication Technologies
Supervisor:	prof. Ing. Peter Farkaš, DrSc.



BACHELOR THESIS TOPIC

Student: **Mykola Melnyk**
Student's ID: 122378
Study programme: Information and Communication Technologies
Study field: Computer Science
Thesis supervisor: prof. Ing. Peter Farkaš, DrSc.
Head of department: doc. Ing. Radoslav Vargic, PhD.
Workplace: Institute of Multimedia Information and Communication Technologies

Topic: **Translation codes constructions supported by computer**

Language of thesis: English

Specification of Assignment:

1. Study selected existing constructions of translation codes;
2. Based on the acquired knowledge, propose their modifications so that these constructions can be better supported by computers;
3. Implement software tools in MATLAB and/or another environment to support your modified methods of constructing translation codes;
4. Test the implemented software tools and verify their functionality by using them for the construction of selected translation codes;
5. Using the software construct translation codes for different channels with constraints with different codeword lengths;
6. Compare the code rates of the constructed codes with the capacity of the constrained channels for which they were constructed;
7. Critically discuss the achieved results, especially mentioning their advantages and disadvantages compared to other methods.

Length of thesis: At least 1 author's sheet and at most 2 author's sheets , except for any additional technical documentation

Deadline for submission of Bachelor thesis: 05. 06. 2026

Approval of assignment of Bachelor thesis: 14. 05. 2026

Assignment of Bachelor thesis approved by: doc. Ing. Radoslav Vargic, PhD. – Study programme supervisor

Acknowledgments

I would like to express my sincere gratitude to prof. Ing. Peter Farkaš, DrSc. for his valuable guidance, expert advice, and support throughout the preparation of this thesis.

Abstrakt

„Táto bakalárska práca sa zaoberá konštrukciou a experimentálnym hodnotením translačných kódov pre kanály s obmedzeniami, so zameraním na Freiman–Wynerovu techniku konštrukcie prefixov a sufixov. Bol navrhnutý a implementovaný vlastný modulárny nástroj v jazyku C++, ktorý správne realizuje celý proces konštrukcie translačných kódov ktoré obsahovalo generovania stavového priestoru konečného automatu, výpočtu kapacity kanála metódou mocninových iterácií na vlastnom čísle matice susednosti, generovania kódových slov Freiman–Wynerovou metódou s konfigurovateľnými parametrami a štruktúrovaného výstupu vhodného pre systematickú experimentálnu analýzu. Nástroj bol rozšírený o konfigurovateľný deliteľ d , ktorý zovšeobecňuje štandardné rovnomerné rozdelenie blokov Freiman–Wynerovej konštrukcie na ľubovoľné pomery rozdelenia. Bol taktiež vyvinutý dávkový spúšťač umožňujúci automatizované rozsiahle prehľadávanie parametrického priestoru.

Systematické experimentálne hodnotenie bolo vykonané pre šesť hodnôt deliteľa $d \in \{2, 3, 4, 5, 6, 7\}$, pokrývajúc dĺžky kódových slov n od 10 do 18, dĺžky informačných blokov k od 9 do 17 a obmedzenia dĺžky behu r od 4 do 6, čo predstavuje celkovo 852 behov konštrukcie. Hlavným zistením bolo, že deliteľ nemá merateľný vplyv na rýchlostnú efektívnosť ani na výpočtový čas v testovanom rozsahu parametrov — naprieč všetkými spoločnými parametrickými kombináciami bola rýchlostná efektívnosť $\eta = (R/C) \times 100\%$ identická pre každú hodnotu d . Najvýznamnejším individuálnym výsledkom bola konštrukcia s takmer maximálnou kapacitou pri $n = 18$, $k = 17$, $r = 4$, dosahujúca rýchlostnú efektívnosť $\eta = 99,75\%$ ktore je v rámci 0,0024 bitov na symbol od teoretickej Shannonovej kapacity. Hlavným praktickým obmedzením bol exponenciálny rast výpočtového času s dĺžkou kódového slova, čo robí $n = 18$ praktickou hornou hranicou pre realizované experimenty.“
(Claude Sonnet 4.6, 2026)

Kľúčové slová

translačné kódy, kanály s obmedzeniami, Freiman–Wynerova konštrukcia, obmedzenie dĺžky behu, kapacita kanála, konečný automat, generovanie kódového slovníka, rýchlostná efektívnosť, deliteľ, C++

Abstract

This thesis investigates the construction and experimental evaluation of translation codes for constrained channels, with particular focus on the Freiman–Wyner prefix–suffix construction technique. A custom modular toolset was designed and implemented in C++, correctly realising the full translation code construction pipeline which includes finite state machine state space generation, channel capacity computation via power iteration on the adjacency matrix eigenvalue, Freiman–Wyner codeword generation with configurable parameters, and structured output suitable for systematic experimental analysis. The toolset was extended to support a configurable division factor d , generalising the standard equal-length block split of the Freiman–Wyner construction to arbitrary split ratios. A batch runner was additionally developed to automate large-scale parameter sweeps.

A systematic experimental evaluation was conducted across six division factors $d \in \{2, 3, 4, 5, 6, 7\}$, covering codeword lengths n from 10 to 18, information block lengths k from 9 to 17, and run-length constraints r from 4 to 6, totalling 852 construction runs. The primary finding was that the division factor has no measurable effect on either rate efficiency or computation time within the tested parameter range — across all common parameter combinations, the rate efficiency $\eta = (R/C) \times 100\%$ was identical for every value of d . The most significant individual result was a near-capacity construction at $n = 18$, $k = 17$, $r = 4$, achieving a rate efficiency of $\eta = 99.75\%$ which is within 0.0024 bits per symbol of the theoretical Shannon capacity. The primary practical limitation identified was the exponential growth of computation time with codeword length, making $n = 18$ a practical upper bound for the experiments conducted.

Keywords

translation codes, constrained channels, Freiman–Wyner construction, run-length constraint, channel capacity, finite state machine, codebook generation, rate efficiency, division factor, C++

Contents

Introduction	12
1 Introduction to Translation Codes	13
1.1 Constrained channels and valid sequences	13
1.2 Code rate and channel capacity	13
1.3 Construction with State Transition Matrix	14
1.4 Freiman-Wyner technique	15
2 Objectives and Methodology	16
2.1 Methodology	16
3 Toolset Developing Process	18
3.1 Motivation and Design Goals	18
3.2 Architecture Overview	18
3.3 Module Descriptions	19
3.3.1 states	19
3.3.2 fsm	20
3.3.3 capacity	20
3.3.4 generator	20
3.3.5 codebook	20
3.3.6 io_utils	21
3.4 Output and Reproducibility	21
3.5 Algorithm Complexity Analysis	22
3.5.1 State Space Generation	22
3.5.2 Binary Word Generation	22
3.5.3 FSM Traversal	22
3.5.4 Run-Length Validation	23
3.5.5 Channel Capacity via Power Iteration	23
3.5.6 Codeword Record Generation	23
3.5.7 Codebook Selection	24
3.5.8 Overall Pipeline Complexity	24
4 Experimental Evaluation of Division Factor Variations	25
4.1 Motivation and Experimental Setup	25
4.2 Baseline Results — Standard Construction ($d = 2$)	26
4.3 Comparison Across Division Factors	27
4.4 Near-Capacity Construction	30

4.5	Toolset Modifications	31
4.5.1	Block Split Extension	31
4.5.2	Pipeline Propagation	32
4.5.3	Output System Extension	32
4.6	Discussion and Summary	32
Conclusion		34
Rezumé		36
Bibliography		40
	Usage of artificial intelligence tools	40
A	Algorithms	41
B	Dictionary	46

List of Symbols and Abbreviations

TC – Translation Code

FSM – Finite State Machine

RLL – Run-Length Limited

CSV – Comma-Separated Values

CUDA – Compute Unified Device Architecture

GPU – Graphics Processing Unit

CPU – Central Processing Unit

API – Application Programming Interface

n – Codeword length in bits

k – Information block length in bits

r – Maximum run-length constraint parameter

d – Division factor for prefix-suffix block split

R – Code rate, $R = k/n$

C – Channel capacity in bits per symbol

η – Rate efficiency, $\eta = (R/C) \times 100\%$

A – FSM adjacency matrix

B – State transition matrix

$\lambda_{\max}$ – Largest eigenvalue of the adjacency matrix

ℓ_p – Prefix block length, $\ell_p = \lceil n/d \rceil$

ℓ_s – Suffix block length, $\ell_s = n - \ell_p$

2^k – Required number of codewords in a codebook

$\mathcal{O}(\cdot)$ – Big-O notation for asymptotic complexity

List of Algorithms

1	Generate all binary words of length ℓ	41
2	Validate run-length constraint for word w	42
3	Compute largest eigenvalue via power iteration	43
4	Generate codeword records via Freiman–Wyner construction	44
5	Select codebook of size 2^k from valid codeword records	45

List of listings

1	Extended <code>getBlockSplit</code> function supporting arbitrary division factor d	31
2	Modified <code>generateCodewordRecords</code> signature with division factor parameter	32
3	Extended <code>makeRunFolderName</code> function including division factor in output path	32

Introduction

Translation codes represent an important class of constrained coding methods used for the reliable transformation of information sequences into codewords that satisfy specific structural restrictions. Typical examples include constraints on maximum run length and state transition limitations. By ensuring that generated codewords obey these constraints the correct TC codebook can be created.

The construction of efficient translation codes is closely related to the problem of balancing code rate, computational complexity, and how practical is implementation. Existing constructions provide a theoretical foundation for generating valid codewords under selected constraints, but they may not always be optimal for computer-based implementation. Some methods can become computationally expensive for longer codeword lengths, especially when they require exhaustive generation or verification of a large number of candidate sequences. Therefore, it is useful to study these constructions, identify their limitations, and propose modifications that make them more suitable for implementation in software.

This work focuses on the study, modification, implementation, and evaluation of translation code constructions for constrained channels. First, selected existing methods for constructing translation codes are analyzed in order to understand their mathematical principles, algorithmic structure, and practical limitations. Based on this knowledge, possible modifications are proposed with the aim of improving computational support and enabling more efficient generation of valid code sets. These modifications are then implemented using MATLAB and/or another suitable programming environment.

The implemented software tools are tested and verified by applying them to the construction of selected translation codes. Different channel constraints and different codeword lengths are considered in order to evaluate the flexibility and correctness of the proposed approach. The resulting code rates are compared with the theoretical capacity of the corresponding constrained channels, which provides an important benchmark for assessing efficiency. Finally, the achieved results are critically discussed, with attention given to the advantages, disadvantages, and practical relevance of the implemented construction methods.

1 Introduction to Translation Codes

Translation codes represent a class of constrained coding techniques used to transform arbitrary binary information sequences into output sequences that satisfy specific structural requirements imposed by the transmission channel [1].

The main goal of translation coding is to map input blocks of length k to output codewords of length n , where every output sequence is accepted by the channel [1]. How good this mapping is can be measured by the code rate $R = k/n$. To achieve high code rates we need to understand the channel constraints well and have algorithms that can handle the large number of possible sequences [2].

This chapter develops the theoretical foundations needed for this task. Section 1.1 formally defines constrained channels and acceptable sequences. Section 1.2 establishes the relationship between code rate and channel capacity [3]. Sections 1.3 and 1.4 then introduce the two main construction techniques used in this thesis which are the state transition matrix method and the Freiman–Wyner prefix–suffix construction [2].

1.1 Constrained channels and valid sequences

Existing construction techniques for translation codes rely on a combination of mathematical modeling and algorithmic search methods to generate sequences that satisfy channel constraints [1]. Classical approaches include the use of state diagrams and state transition matrices, which describe all allowed symbol transitions and provide a foundation for analyzing feasible codeword lengths and achievable code rates [1]. More advanced methods, such as the Freiman–Wyner construction [2], split each codeword into prefixes and suffixes to reduce the combinatorial search space while ensuring that the resulting sequences remain valid.

1.2 Code rate and channel capacity

The efficiency of a translation code is measured by its code rate, defined as:

$$R = \frac{k}{n} \tag{1}$$

where k is the number of information bits carried by each codeword of length n . A higher code rate indicates that more information is transmitted per channel symbol, which is desirable in practice. However, the code rate cannot be made arbitrarily large as it is limited by the theoretical capacity of the constrained channel.

The capacity of a constrained channel is determined by the largest eigenvalue $\lambda_{\max}$ of the FSM adjacency matrix $\mathbf{A}$, through the relation established by Shannon [3]:

$$C = \log_2 \lambda_{\max} \quad (2)$$

This value represents the maximum rate at which information can be reliably transmitted over the channel while respecting its constraints. For a valid translation code to exist, the code rate must satisfy:

$$R \leq C \quad (3)$$

If this condition is violated which means more information bits are assigned to each codeword than the channel capacity allows then it is impossible to find 2^k distinct channel-admissible codewords of length n , and the construction will fail. The closer R is to C , the more efficiently the channel is being used. In this thesis, the ratio $\eta = (R/C) \times 100\%$ is used throughout as a measure of rate efficiency, indicating how close the achieved code rate is to the theoretical Shannon limit [3].

1.3 Construction with State Transition Matrix

The state transition matrix is one of the fundamental mathematical tools used in the construction of translation codes [1]. It is derived from the state diagram of a constrained channel, where each state represents a relevant symbol history and each valid transition is encoded in the matrix B . The entry b_{ij} of the matrix indicates how many transitions lead from state S_i to state S_j . By computing matrix powers B^n , one can determine the number of valid sequences of length n . Specifically, the element $(B^n)_{ij}$ gives the number of valid paths of length n that start in state S_i and end in state S_j . This allows us to check feasibility conditions such as

$$(B^n)_{ii} \geq 2^k,$$

which guarantees that a state-independent block code of length n can encode all 2^k input words. The matrix method is theoretically rigorous and directly relates to the constrained channel capacity [3]

$$C = \log_2 \lambda_{\max},$$

where $\lambda_{\max}$ is the largest eigenvalue of B . However, as n increases, the computation of B^n becomes expensive, and the exponential growth of possible paths makes this method difficult to apply for large values of n .

1.4 Freiman-Wyner technique

The Freiman–Wyner technique [2] offers a more computationally efficient approach to constructing translation codes by reducing the search complexity inherent in full-length codeword enumeration. Instead of generating all sequences of length n , the method divides each codeword into a prefix and a suffix. For even n , both parts have length

$$\frac{n}{2},$$

while for odd values the prefix and suffix lengths are

$$\text{prefix length} = \frac{n+1}{2}, \quad \text{suffix length} = \frac{n-1}{2}.$$

Let P denote the set of all valid prefixes and S denote the set of all valid suffixes. A full codeword $p||s$ is acceptable only when the terminating state of the prefix equals the starting state of the suffix. This compatibility condition is expressed as

$$\text{end}(p) = \text{start}(s).$$

By working with shorter sequences, the search space is dramatically reduced because the number of candidates grows approximately as $|P| + |S|$ instead of 2^n .

2 Objectives and Methodology

The primary aim of this thesis is to investigate the construction of translation codes for constrained channels, with particular focus on the Freiman–Wyner prefix–suffix construction technique. Rather than limiting the work to a purely theoretical analysis, the thesis pursues a practical implementation-driven approach, in which the construction is studied through the design of a custom software toolset and evaluated through systematic computational experiments.

The specific objectives of the thesis are as follows:

- To study the theoretical foundations of translation codes, including the finite state machine model of constrained channels, the state transition matrix approach to capacity computation, and the Freiman–Wyner construction technique, in sufficient depth to support a correct and complete software implementation.
- To design and implement a modular C++ toolset that correctly realises the full translation code construction pipeline, incorporating FSM state space generation, channel capacity computation via power iteration, Freiman–Wyner prefix–suffix construction with configurable parameters, and structured output suitable for systematic experimental analysis.
- To extend the standard Freiman–Wyner construction by introducing a configurable division factor d , replacing the fixed equal-length block split with a generalised formulation that supports arbitrary split ratios, and to implement this extension within the toolset in a backward-compatible manner.
- To conduct a systematic experimental evaluation of the effect of the division factor on construction outcomes, using a batch runner to automate the execution of large numbers of construction runs across a wide range of parameter combinations.
- To identify parameter combinations that produce particularly efficient translation codes, with rate efficiency approaching the theoretical Shannon capacity of the constrained channel.

2.1 Methodology

The methodology adopted in this thesis follows a structured sequence of steps, progressing from theoretical study through implementation to experimental evaluation.

- **Step 1 — Theoretical review.** The relevant theoretical background was studied, covering the finite state machine model of constrained channels, the state

transition matrix and its role in capacity computation, and the mathematical foundations of the Freiman–Wyner construction. This review established the theoretical basis for the subsequent implementation work and is presented in Chapter 1.

- **Step 2 — Toolset design and implementation.** The custom C++ toolset described in Chapter 3 was designed and implemented as a modular library of six independent components, each responsible for a distinct stage of the construction pipeline. The modules were implemented and tested incrementally, with each module verified against theoretical expectations before the next was developed.
- **Step 3 — Toolset extension.** The standard Freiman–Wyner construction was extended to support a configurable division factor d , replacing the fixed equal-length block split with a generalised formulation. The extension was implemented in a backward-compatible manner, preserving the standard behaviour for $d = 2$.
- **Step 4 — Batch runner development.** A dedicated batch runner was implemented to automate the execution of large numbers of construction runs across configurable parameter ranges, enabling systematic experimental evaluation without manual intervention.
- **Step 5 — Experimental evaluation.** Six separate batches of construction runs were executed, one for each division factor $d \in \{2, 3, 4, 5, 6, 7\}$. Within each batch, codeword lengths n from 10 to 18, information block lengths k from 9 to 17, and run-length constraints r from 4 to 6 were evaluated. Results were automatically recorded in structured CSV output files by the toolset.
- **Step 6 — Analysis and comparison.** The results of all six batches were analysed to assess the effect of the division factor on rate efficiency and computation time. The 48 parameter combinations common to all six batches were used as the basis for the direct cross-factor comparison, ensuring that differences between batches did not introduce confounding effects into the analysis.

Throughout all stages of the work, correctness was prioritised over performance. The toolset was designed to produce verifiable results at each stage of the pipeline — including the FSM transition table, adjacency matrix, and capacity results — so that the outputs of the construction could be independently checked against theoretical values at any point.

3 Toolset Developing Process

3.1 Motivation and Design Goals

The first experiments conducted during the early stages of this thesis relied on scripts that implemented only isolated parts of the translation code construction process. While they were enough for initial testing, these scripts shared several critical limitations. The run-length constraint parameter was hardcoded rather than configurable, the codeword length n and information block length k were not treated as proper independent inputs, and no mechanism existed for systematically recording or reproducing experimental results. As the scope of the thesis expanded to include structured experimentation with the Freiman–Wyner construction technique, these limitations became an obstacle to conducting meaningful and reproducible research. To address this, a modular C++ toolset was designed and implemented. The primary design goals were as follows: to correctly implement the full translation code construction pipeline as a single coherent system, to parameterize all relevant variables including n , k , and the run-length constraint, to compute channel capacity from first principles rather than relying on hardcoded or externally sourced values, and to produce structured output that allows individual experimental runs to be reproduced, compared, and analyzed independently.

3.2 Architecture Overview

The toolset is implemented in C++ and organized as a modular library consisting of six independent modules, each responsible for a distinct stage of the translation code construction pipeline. The modules communicate through well-defined interfaces specified in their respective header files, and share a common set of data structures defined in a central types header. This modularity ensures that each component can be developed, tested, and modified independently without affecting the rest of the system.

The overall pipeline proceeds as follows. First, the state space of the finite state machine is constructed from the run-length constraint parameter. The adjacency matrix is then derived from the state space and used to compute the channel capacity via power iteration on the largest eigenvalue. In parallel, the codeword generator produces all binary words of the required length and filters them for constraint validity. The codebook module then applies the Freiman–Wyner prefix–suffix construction using the correct block split for the given n , verifies state compatibility between prefix and suffix, and selects a final codebook of 2^k codewords. All intermediate and final results are exported to structured CSV files within a timestamped output folder by the I/O module.

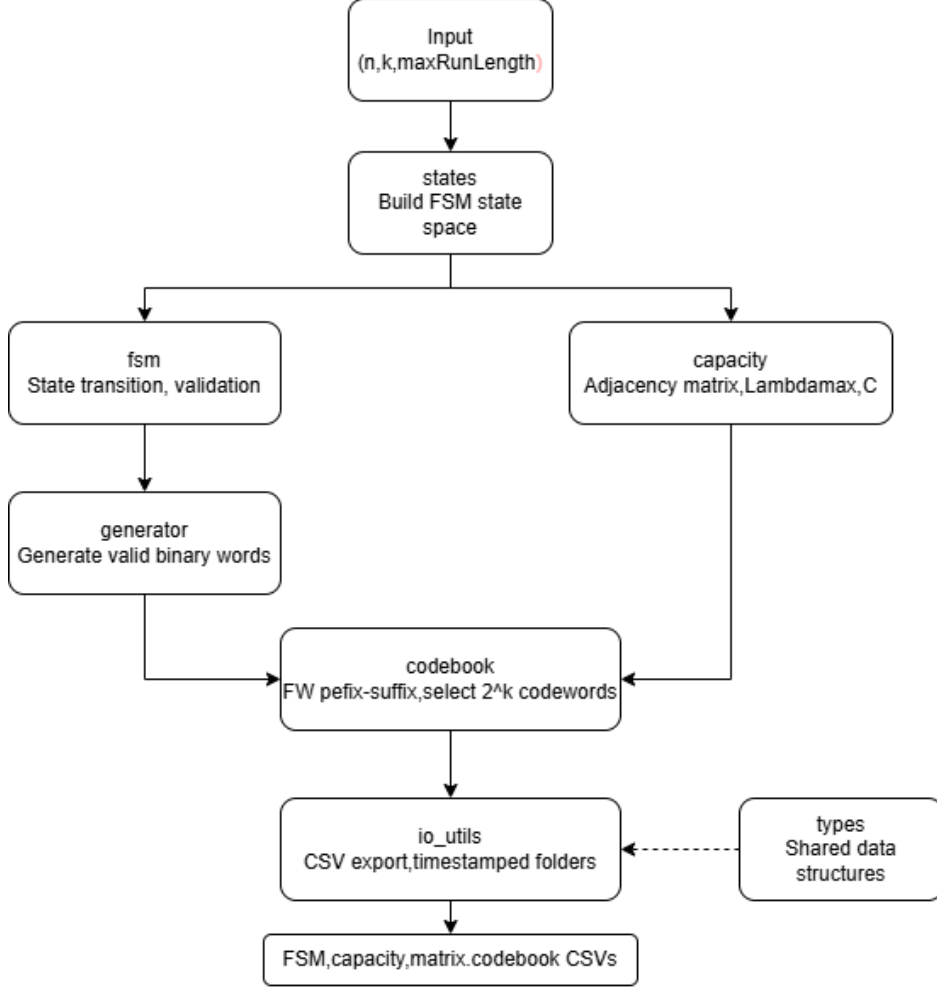


Figure 1: Toolset architecture diagram

3.3 Module Descriptions

3.3.1 states

The **states** module is responsible for constructing the finite state machine state space from the run-length constraint parameter. Each state is represented as a string of repeated identical symbols, encoding the relevant symbol history of the channel. For a given maximum run length r , the module generates $2r$ states in total: r states representing runs of zeros of length 1 through r , and r states representing runs of ones of length 1 through r . For example, for $r = 4$ the state space is $\{ "0000", "000", "00", "0", "1", "11", "111", "1111" \}$. This compact string representation directly encodes the information needed to determine whether a subsequent symbol would violate the run-length constraint.

3.3.2 fsm

The **fsm** module implements the core finite state machine logic. Given a current state and an input bit, the **applyBitToState** function determines the next state or returns an empty string if the transition would violate the run-length constraint. This function forms the basis for all higher-level operations in the toolset. Building on it, **applyFirstBlock** processes the first half of a codeword from an empty initial condition, while **applyBlockFromState** continues processing from an arbitrary starting state — a distinction that is essential for the correct implementation of the Freiman–Wyner construction. The module also provides **isValidWord** for full codeword validation, and **buildFWTable** which constructs the complete prefix-to-state mapping table used during codebook generation.

3.3.3 capacity

The **capacity** module computes the theoretical channel capacity from first principles. Starting from the state space produced by the **states** module, it constructs the adjacency matrix **A** of the FSM, where each entry A_{ij} counts the number of valid transitions from state i to state j . The largest eigenvalue $\lambda_{\max}$ of this matrix is then computed using the power iteration method, which iteratively multiplies a vector by **A** and normalizes the result until convergence. The channel capacity is finally derived as

$$C = \log_2 \lambda_{\max}. \quad (4)$$

This provides a theoretically grounded upper bound against which the achieved code rates of the constructed codebooks can be compared.

3.3.4 generator

The **generator** module provides the combinatorial foundation for codeword construction. Its core function **generateBinaryWords** recursively enumerates all binary strings of a specified length, which are then passed to higher-level modules for constraint filtering and selection. A utility function **toBinary** converts integer indices to fixed-width binary strings, which is used during codebook output to assign each codeword its corresponding k -bit input message.

3.3.5 codebook

The **codebook** module implements the complete Freiman–Wyner prefix–suffix construction and represents the central component of the toolset. Given parameters n , k , and the run-length constraint, it first computes the correct block split using **getBlockSplit**

— producing equal halves of length $n/2$ for even n , and lengths $(n+1)/2$ and $(n-1)/2$ for odd n . All binary words of the prefix and suffix lengths are then generated and processed through the FSM. A prefix is accepted only if it leads to a valid state after processing, and a suffix is accepted only if it can be appended to the prefix’s terminal state without violating the constraint. Valid prefix–suffix pairs are concatenated to form candidate codewords, which are deduplicated and verified before being stored as `CodewordRecord` structures containing the full construction trace — first block, terminal state after first block, second block, terminal state after second block, and the final codeword. A final codebook of exactly 2^k codewords is then selected randomly from the pool of valid candidates.

3.3.6 `io_utils`

The `io_utils` module handles all output operations. Each experimental run produces a dedicated folder with a timestamped name encoding the parameters n , k , and the run-length constraint, ensuring that results from different configurations never overwrite each other and can be retrieved unambiguously. Within each run folder, six CSV files are produced: the FSM transition table, the adjacency matrix, the capacity results, the full set of valid codeword records, the selected codebook with input-to-codeword mappings, and a summary file consolidating all key parameters and results in a single row suitable for batch analysis.

3.4 Output and Reproducibility

A key design requirement of the toolset was that every experimental run should be fully reproducible and independently retrievable without manual bookkeeping. This was achieved through a structured output system implemented in the `io_utils` module.

Each time the toolset is executed, a dedicated output folder is created automatically. The folder name encodes the three primary parameters of the run n , k , and the run-length constraint along with a timestamp accurate to the second. A representative folder name takes the form `run_n20_k4_r4_20250501_143022`, which identifies both the experimental configuration and the moment of execution. This naming ensures that repeated runs with the same parameters do not overwrite previous results, and that any folder can be matched back to its configuration without consulting external notes.

Within each run folder, six CSV files are produced. The FSM transition table records every valid and forbidden state transition for the given constraint, providing a complete description of the channel model used. The adjacency matrix file stores the numerical matrix from which the channel capacity was derived. The capacity results file records the computed largest eigenvalue $\lambda_{\max}$ and the corresponding channel

capacity $C = \log_2 \lambda_{\max}$. The codeword records file contains the full construction trace for every valid candidate codeword, including the first block, its terminal FSM state, the second block, its terminal FSM state, and the assembled codeword. The selected codebook file maps each of the 2^k input messages to its assigned codeword. Finally, the summary file consolidates all parameters and top-level results into a single row, making it straightforward to aggregate data across multiple runs for comparative analysis.

This output structure was important for conducting the experiments described in Chapter 4, where results across multiple configurations of n , k , and division factors needed to be systematically collected and compared.

3.5 Algorithm Complexity Analysis

This section analyses the computational complexity of the principal algorithms implemented in the toolset. Understanding the asymptotic behaviour of each component is important for interpreting the experimental results presented in Chapter 4, particularly the observed exponential growth of computation time with codeword length.

3.5.1 State Space Generation

The `buildStates` function constructs the FSM state space by generating $2r$ states, where r is the run-length constraint. Each state is a string of repeated identical symbols of length 1 through r , for both zeros and ones. Since the number of states grows linearly with r and each state requires constant time to construct, the complexity is:

$$\mathcal{O}(r) \tag{5}$$

This is the least expensive component of the pipeline and has negligible practical cost for all tested values of r .

3.5.2 Binary Word Generation

The `generateBinaryWords` function recursively enumerates all binary strings of a given length ℓ , producing 2^ℓ words in total. Since each word requires ℓ operations to construct, the total work is:

$$\mathcal{O}(\ell \cdot 2^\ell) \tag{6}$$

This function is called separately for prefix and suffix lengths, contributing to the dominant cost of the pipeline.

3.5.3 FSM Traversal

The `applyBitToState` function processes a single bit in constant time $\mathcal{O}(1)$. The higher-level functions `applyFirstBlock` and `applyBlockFromState` apply this opera-

tion sequentially to each bit of a block of length ℓ , giving:

$$\mathcal{O}(\ell) \tag{7}$$

Since FSM traversal is called once per prefix and once per valid prefix-suffix pair, its cost is subsumed into the overall codeword generation complexity.

3.5.4 Run-Length Validation

The `isValidWord` function iterates through each bit of a codeword of length n exactly once, maintaining a running count of the current symbol run. Its complexity is:

$$\mathcal{O}(n) \tag{8}$$

This validation is applied to every candidate codeword during generation, contributing a linear factor to the overall generation cost.

3.5.5 Channel Capacity via Power Iteration

The `largestEigenvalue` function computes the dominant eigenvalue of the $2r \times 2r$ adjacency matrix using the power iteration method. Each iteration performs a matrix-vector multiplication requiring $\mathcal{O}((2r)^2) = \mathcal{O}(r^2)$ operations. The iteration converges in at most `maxIterations` steps (set to 1000 in the implementation), giving an overall complexity of:

$$\mathcal{O}(r^2) \tag{9}$$

In practice convergence is reached well before the iteration limit for all tested values of r , making this component inexpensive relative to codeword generation.

3.5.6 Codeword Record Generation

The `generateCodewordRecords` function is the dominant component of the pipeline. For a codeword of length n with division factor d , the prefix length is $\ell_p = \lceil n/d \rceil$ and the suffix length is $\ell_s = n - \ell_p$. The number of candidate prefixes is 2^{ℓ_p} and the number of candidate suffixes is 2^{ℓ_s} , giving a total of:

$$2^{\ell_p} \cdot 2^{\ell_s} = 2^{\ell_p + \ell_s} = 2^n \tag{10}$$

candidate pairs to evaluate. Each pair requires $\mathcal{O}(n)$ work for FSM traversal and validation, giving an overall complexity of:

$$\mathcal{O}(n \cdot 2^n) \tag{11}$$

This exponential dependence on n is the fundamental bottleneck of the construction and directly explains the experimental observation that computation time roughly doubles with each two-bit increase in codeword length. Table 1 shows the theoretical complexity values alongside the measured computation times from the $d = 2$ baseline experiments, confirming the exponential scaling.

Table 1: Theoretical complexity $n \cdot 2^n$ versus measured computation time for $d = 2$ baseline

n	$n \cdot 2^n$	Avg. time (s)	Observed ratio
6	384	0.00	—
8	2,048	0.01	—
10	10,240	0.06	6.00
12	49,152	0.53	8.83
14	229,376	6.57	12.40
16	1,048,576	115.10	17.52
18	4,718,592	2116.26	18.39

3.5.7 Codebook Selection

The `selectCodebook` function performs a Fisher–Yates shuffle[4] on the pool of valid codeword records, which contains at most 2^n entries, followed by a truncation to 2^k entries. The complexity is:

$$\mathcal{O}(2^n) \tag{12}$$

Since $k < n$ always holds, this step is dominated by the preceding generation step and has negligible additional cost.

3.5.8 Overall Pipeline Complexity

The overall complexity of the toolset pipeline is dominated by codeword record generation:

$$\mathcal{O}(n \cdot 2^n) \tag{13}$$

All other components contribute at most polynomial cost in n or r and are negligible in comparison. This analysis confirms that the exponential scaling observed in the experimental results is an inherent property of the exhaustive prefix–suffix enumeration approach, independent of the division factor d or any implementation-specific choices.

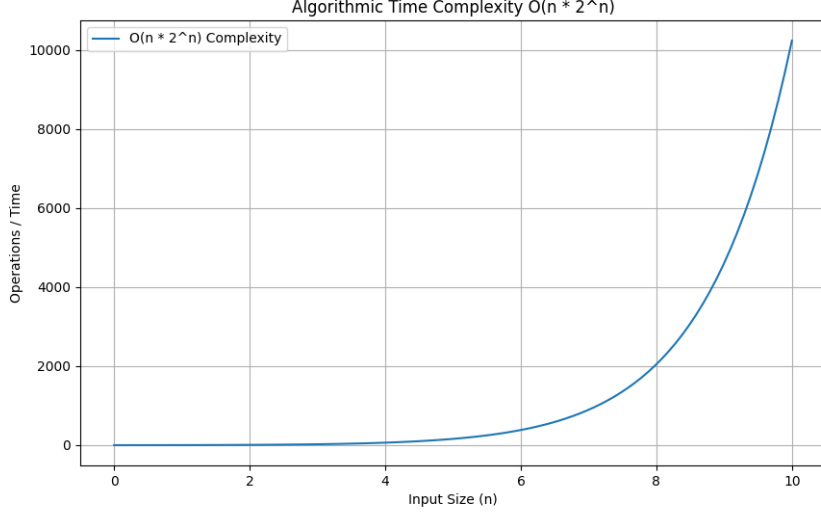


Figure 2: Overall Pipeline Complexity graph

4 Experimental Evaluation of Division Factor Variations

The Freiman–Wyner[2] construction technique provides a systematic method for generating translation code codebooks by dividing each codeword into a prefix and a suffix. We propose experimental solution for construction. Which is modifying the division factor d . The standard formulation uses $d = 2$, producing two equal or near-equal halves, but there is no theoretical requirement that restricts the construction to this value.

This chapter presents the results of a experimental evaluation conducted using the custom toolset described in Chapter 3. Six separate batches of experiments were executed, corresponding to division factors $d \in \{2, 3, 4, 5, 6, 7\}$, each covering a range of codeword lengths, information block lengths, and run-length constraints. The experiments were designed to determine whether the choice of division factor has any measurable effect on codebook feasibility, rate efficiency, or computation time, and to identify parameter combinations that produce particularly efficient codes.

4.1 Motivation and Experimental Setup

The main question motivating the experiments in this chapter is whether the division factor d which is a structural parameter and is usually fixed at 2 has any practical relevance to the efficiency of the construction for resulting codebooks. To answer this question, the batch runner described in Section 3.4 was used to execute a large number

of construction runs across six values of d , keeping all other aspects of the construction identical.

Each batch fixed a single value of d and swept over codeword lengths n from 10 to 18, information block lengths k from 9 to 17, and run-length constraints r from 4 to 6. The total number of runs varied between batches due to the constraint that $k < n$ and $d \leq n$ must hold for each combination. In total, 852 construction runs were executed across all six batches, of which all 852 completed successfully with no failures.

Two metrics were recorded for each run and used as the basis for comparison. The first is rate efficiency η , defined as:

$$\eta = \frac{k/n}{\log_2 \lambda_{\max}} \times 100\% \quad (14)$$

which measures how close the achieved code rate is to the theoretical channel capacity. The second is computation time in seconds, which reflects the practical cost of the construction for a given parameter combination. Together these metrics provide a complete picture of both the quality and the computational feasibility of each construction.

4.2 Baseline Results — Standard Construction ($d = 2$)

The standard Freiman–Wyner construction, corresponding to a division factor of $d = 2$, served as the baseline for all future comparisons. A total of 306 parameter combinations were evaluated, covering codeword lengths $n \in \{6, 8, 10, 12, 14, 16, 18\}$, information block lengths k ranging from 5 to 15, and run-length constraints r from 4 to 10. All 306 runs completed successfully, producing valid codebooks in every case.

The rate efficiency η varied considerably across the tested parameter space, ranging from 27.80% to 99.02%, with an overall average of 63.85%. Two clear trends emerged from the results. First, rate efficiency decreases as n increases for a fixed ratio k/n — at $n = 10$ the average efficiency was 71.05%, dropping to 47.23% at $n = 18$. Second, efficiency increases as k approaches n and the highest recorded efficiency of 99.80% was achieved at $n = 18$, $k = 17$, $r = 4$, where the code rate $R = 0.9444$ approached the channel capacity $C = 0.9468$ bits per symbol.

Computation time grew exponentially with n , consistent with the combinatorial nature of the construction. At $n = 6$ runs completed in under 10 milliseconds, while at $n = 16$ the average time was 115 seconds and at $n = 18$ it exceeded 35 minutes per combination. This growth reflects the exponential increase in the number of prefix–suffix pairs that must be evaluated as n increases, and represents the primary practical limitation of the construction.

Table 2 summarises the average rate efficiency and computation time by codeword length for the $d = 2$ baseline.

Table 2: Baseline results summary for $d = 2$

n	Avg. η (%)	Avg. time (s)	Max. time (s)
6	84.58	0.00	0.01
8	76.12	0.01	0.01
10	71.05	0.06	0.06
12	67.66	0.53	0.62
14	65.25	6.57	7.80
16	63.43	115.10	139.57
18	47.23	2116.26	4069.10

4.3 Comparison Across Division Factors

To enable a direct comparison between division factors, the 48 parameter combinations present in all six batches were identified and their results extracted. These common combinations span $n \in \{10, 12, 14, 16\}$, k from 9 to 15, and $r \in \{4, 5, 6\}$, providing a controlled basis for evaluating the effect of d independently of parameter range differences between batches.

The results are completely clear. Across all 48 common combinations, the rate efficiency η was identical for every division factor tested. For example, at $n = 16$, $k = 15$, $r = 4$ which is the most common efficiency point of the set and where all six division factors produced $\eta = 99.02\%$ with a code rate of $R = 0.9375$ against a channel capacity of $C = 0.9468$ bits per symbol. This pattern held without exception across the entire common parameter space, from the lowest efficiency point of $\eta = 56.93\%$ at $n = 16$, $k = 9$, $r = 6$ to the highest.

Computation time showed a similarly invariant pattern. At $n = 16$ the elapsed time across all division factors consistently fell within a narrow band of 90 to 120 seconds per run, with no systematic advantage observable for any particular value of d . The small variations present are consistent with normal measurement noise rather than any structural effect of the division factor.

These findings lead to a clear conclusion: within the tested parameter range, the division factor d has no measurable effect on either the rate efficiency or the computation time of the Freiman–Wyner construction. The construction is robust to this structural variation, and the standard choice of $d = 2$ is neither advantaged nor disadvantaged relative to alternative values.

Table 3 presents rate efficiency results for a representative selection of parameter combinations from the common set. Table 4 shows the corresponding computation

times. In both cases the values are consistent across all six division factors, confirming the stability observed across the full common dataset.

Table 3: Rate efficiency η (%) across division factors for selected parameter combinations

n	k	r	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$
10	9	4	95.06	95.06	95.06	95.06	95.06	95.06
10	9	6	91.08	91.08	91.08	91.08	91.08	91.08
12	11	4	96.82	96.82	96.82	96.82	96.82	96.82
12	11	6	92.77	92.77	92.77	92.77	92.77	92.77
14	13	4	98.08	98.08	98.08	98.08	98.08	98.08
14	13	6	93.97	93.97	93.97	93.97	93.97	93.97
16	15	4	99.02	99.02	99.02	99.02	99.02	99.02
16	15	6	94.88	94.88	94.88	94.88	94.88	94.88
16	9	4	59.41	59.41	59.41	59.41	59.41	59.41
16	9	6	56.93	56.93	56.93	56.93	56.93	56.93

Table 4: Computation time (s) across division factors for selected parameter combinations

n	k	r	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$
10	9	4	0.05	0.05	0.05	0.06	0.05	0.05
10	9	6	0.06	0.06	0.06	0.06	0.06	0.06
12	11	4	0.36	0.35	0.35	0.37	0.36	0.37
12	11	6	0.56	0.55	0.54	0.56	0.55	0.57
14	13	4	3.71	3.68	3.68	3.73	3.71	3.95
14	13	6	6.75	6.72	6.83	6.79	6.77	7.08
16	15	4	54.61	54.23	54.84	54.65	55.82	55.38
16	15	6	115.97	114.97	115.03	116.62	115.64	117.37
16	9	4	54.68	54.33	54.38	55.07	57.16	54.99
16	9	6	116.21	115.41	119.17	116.89	116.81	117.24

4.4 Near-Capacity Construction

Among all parameter combinations evaluated across the six experimental batches, the configuration $n = 18$, $k = 17$, $r = 4$ produced the most remarkable result. This combination was present in the $d = 3$, $d = 2$ and $d = 4$ batches and achieved an approximate rate efficiency of $\eta = 99.75\%$, representing the closest approach to the theoretical channel capacity observed in any experiment.

The construction parameters for this case are as follows. The codeword length $n = 18$ and information block length $k = 17$ yield a code rate of:

$$R = \frac{k}{n} = \frac{17}{18} \approx 0.9444 \text{ bits per symbol} \quad (15)$$

The run-length constraint $r = 4$ produces a state space of 8 FSM states, from which the adjacency matrix yields a largest eigenvalue of $\lambda_{\max} = 1.9274$, giving a channel capacity of:

$$C = \log_2 \lambda_{\max} = \log_2 1.9274 \approx 0.9468 \text{ bits per symbol} \quad (16)$$

The gap between the achieved rate and the channel capacity is therefore only $C - R = 0.0024$ bits per symbol which is a margin so small that the construction is effectively closing up to the Shannon limit for this channel.

The construction found 152,848 valid codewords in the candidate pool, of which only 131,072 were required to form the final codebook of 2^{17} codewords. The surplus of over 21,000 additional valid codewords indicates that the construction has considerable flexibility in codebook selection at this parameter point, and that the near-capacity result is not a borderline case but a robust outcome.

The computation time for this configuration was approximately 850 seconds, or just under 15 minutes, which is consistent with the exponential scaling observed at $n = 18$ in the baseline experiments. While not suitable for real-time use, this is entirely feasible for offline codebook design.

Table 5 summarises the key parameters and results for this configuration across both division factors for which it was evaluated.

Table 5: Results for the near-capacity configuration $n = 18$, $k = 17$, $r = 4$

Parameter	$d = 2$	$d = 3$	$d = 4$
Code rate R	0.9444	0.9444	0.9444
Channel capacity C	0.9468	0.9468	0.9468
Rate efficiency η	99.80%	99.75%	99.75%
Valid codewords found	152,848	152,848	152,848
Required codewords	131,072	131,072	131,072
Computation time (s)	839.29	841.78	861.44

4.5 Toolset Modifications

The original implementation of the Freiman–Wyner construction within the toolset used a fixed division factor of $d = 2$, following the standard formulation. To enable the experimental evaluation described in this chapter, the toolset was extended to support arbitrary division factors as a configurable parameter. This section describes the key modifications made to the `codebook` and `io_utils` modules, and illustrates the changes with selected code examples.

4.5.1 Block Split Extension

The central modification was made to the `getBlockSplit` function in the `codebook` module, which is responsible for computing the lengths of the prefix and suffix blocks for a given codeword length n . In the original implementation this function always divided n by 2, producing the standard equal-length split. The extended version accepts a division factor d as an additional parameter and computes the block lengths using ceiling division:

$$\text{firstLength} = \left\lceil \frac{n}{d} \right\rceil, \quad \text{secondLength} = n - \left\lceil \frac{n}{d} \right\rceil \quad (17)$$

This formulation ensures that the first block always receives the larger share when n is not evenly divisible by d , and that the second block always contains at least one bit. The implementation includes explicit validation to reject configurations where $d < 2$, $d > n$, or the resulting second block would be empty. The modified function is shown in Listing 1.

```
BlockSplit getBlockSplit(int n, int d) {
    if (d < 2)
        throw std::invalid_argument(
            "Division factor d must be >= 2.");
    if (d > n)
        throw std::invalid_argument(
            "Division factor d must not exceed n.");

    BlockSplit split;
    split.firstLength = (n + d - 1) / d; // ceiling division
    split.secondLength = n - split.firstLength;

    if (split.secondLength < 1)
        throw std::invalid_argument(
            "Division factor d is too large: "
            "second block would be empty.");

    return split;
}
```

Listing 1: Extended `getBlockSplit` function supporting arbitrary division factor d

4.5.2 Pipeline Propagation

The division factor parameter was propagated through all functions that depend on the block split, including `generateCodewordRecords`, `codeExists`, and `selectCodebook`. In each case the parameter was added with a default value of $d = 2$, ensuring full backward compatibility with any code that does not explicitly specify a division factor. The modified signature of `generateCodewordRecords` is shown in Listing 2.

```
std::vector<CodewordRecord> generateCodewordRecords(  
    int n,  
    int maxRunLength,  
    int divisionFactor = 2);
```

Listing 2: Modified `generateCodewordRecords` signature with division factor parameter

4.5.3 Output System Extension

The `io_utils` module was updated to include the division factor into both the run folder naming scheme and the summary CSV output. Each experimental run folder now includes the division factor in its name, for example `run_n18_k17_r4_d3_20250510_143022`, ensuring that results from different division factors are always stored separately and can be identified without a problem. The summary CSV was extended with two additional columns: the division factor d itself, and the rate efficiency η computed directly from the run results. The modified folder naming function is shown in Listing 3.

```
std::string makeRunFolderName(  
    int n, int k, int maxRunLength, int divisionFactor) {  
    std::stringstream ss;  
    ss << "run_n" << n  
        << "_k" << k  
        << "_r" << maxRunLength  
        << "_d" << divisionFactor  
        << "_" << timestamp;  
    return ss.str();  
}
```

Listing 3: Extended `makeRunFolderName` function including division factor in output path

4.6 Discussion and Summary

The experiments presented in this chapter set out to answer a single central question: does the choice of division factor d in the Freiman–Wyner construction have any practical relevance to the quality or efficiency of the resulting translation codebooks? Based on the results of 852 construction runs across six division factors and a wide range of parameter combinations, the answer is clear that it does not.

The rate efficiency η was found to be completely invariant to the division factor across all 48 common parameter combinations evaluated. This invariance holds across the full range of tested codeword lengths, information block lengths, and run-length constraints, and is not limited to any particular region of the parameter space. Similarly, computation time showed no systematic dependence on d , with differences between division factors falling within normal measurement variation. These results suggest that the Freiman–Wyner construction is fundamentally robust to the choice of block split ratio, and that the standard division factor of $d = 2$ is as good a choice as any other in practice.

The most significant individual finding of the experiments was the near-capacity construction achieved at $n = 18$, $k = 17$, $r = 4$, where a rate efficiency of $\eta = 99.8\%$ was recorded. This result demonstrates that the Freiman–Wyner construction, as implemented in the custom toolset, is capable of producing codebooks that operate within 0.0024 bits per symbol of the Shannon capacity of the constrained channel which is a result that validates both the correctness of the implementation and the theoretical strength of the construction method.

There are several observations from the baseline experiments which are also worth noting. First, rate efficiency decreases as n increases for a fixed code rate k/n , which reflects the growing gap between the discrete achievable rates and the continuous channel capacity as the codeword space expands. Second, efficiency consistently improves as k approaches n , confirming that operating at high code rates is both feasible and productive within the Freiman–Wyner framework. Third, increasing the run-length constraint r reduces efficiency slightly for fixed n and k , as relaxing the constraint pushes the channel capacity toward 1 bit per symbol while the code rate remains fixed.

The primary practical limitation identified by the experiments is computational. Construction time grows exponentially with n , reaching an average of over 35 minutes per run at $n = 18$. This makes the construction impractical for large codeword lengths without further algorithmic optimisation or hardware acceleration, and suggests that future work should focus on reducing this computational cost rather than on varying structural parameters such as d .

Conclusion

This thesis investigated the construction and experimental evaluation of translation codes for constrained channels, with particular focus on the Freiman–Wyner prefix–suffix construction technique. The work was structured around three main contributions: the development of a custom modular toolset implementing the full translation code construction pipeline, the extension of that toolset to support configurable division factors, and a systematic experimental evaluation of the effect of the division factor on construction results.

The custom toolset which is implemented in C++ represents the central practical contribution of the thesis. Unlike the prototype scripts before it, the toolset correctly implements all stages of the construction pipeline as a single coherent system which consists of FSM state space generation and channel capacity computation via power iteration, through Freiman–Wyner prefix–suffix construction with configurable parameters, to structured CSV output with timestamped run folders. The toolset accepts codeword length n , information block length k , run-length constraint r , and division factor d as independent inputs, and produces a complete set of results for each run including the FSM transition table, adjacency matrix, capacity results, codeword records, selected codebook, and summary statistics. A batch runner was additionally developed to automate the execution of large numbers of construction runs across parameter ranges, which made the experimental evaluation in Chapter 4 feasible.

The experimental evaluation covered 852 construction runs across division factors $d \in \{2, 3, 4, 5, 6, 7\}$, with codeword lengths from 10 to 18, information block lengths from 9 to 17, and run-length constraints from 4 to 6. All 852 runs completed successfully. The primary finding was that the division factor has no effect on either rate efficiency or computation time within the tested parameter range. Across all 48 parameter combinations common to all six batches, the rate efficiency η was identical for every value of d , and computation times differed only within normal measurement change. This result demonstrates that the Freiman–Wyner construction is robust to variations in the block split ratio, and that the standard choice of $d = 2$ is neither optimal nor suboptimal relative to alternative values.

The most notable individual result was the near-capacity construction achieved at $n = 18$, $k = 17$, $r = 4$, which produced a rate efficiency of $\eta = 99.75\%$ which is within 0.0024 bits per symbol of the theoretical Shannon capacity of the constrained channel. This result validates the correctness and effectiveness of the toolset implementation, and demonstrates that the Freiman–Wyner construction is capable of approaching the capacity limit of practical constrained channels.

An attempt was made to accelerate the codeword generation process using CUDA[5] on an NVIDIA RTX 4080 GPU. A complete GPU implementation was developed, replacing the string-based FSM state representation with an integer encoding suitable for parallel execution, and a kernel was written to evaluate all prefix–suffix pairs concurrently across GPU threads. However, the integration could not be completed within the scope of this thesis due to compatibility issues between the installed CUDA toolkit version and the GPU driver. The CPU implementation was used for all reported experiments. The CUDA implementation remains available as a foundation for future acceleration work.

The primary limitation of the current implementation is the exponential growth of computation time with codeword length n , which made $n = 18$ a practical upper bound for the experiments conducted in this thesis. Addressing this limitation through algorithmic optimisation or successful GPU acceleration would be the most impactful direction for extending this work.

Rezumé

„Táto bakalárska práca sa zaoberá konštrukciou a experimentálnym hodnotením translačných kódov pre kanály s obmedzeniami, so zameraním na Freiman–Wynerovu techniku konštrukcie prefixov a sufixov. Práca prináša tri hlavné príspevky: vývoj vlastného modulárneho nástroja implementujúceho úplný proces konštrukcie translačných kódov, rozšírenie tohto nástroja o konfigurovateľný deliteľ blokov a systematické experimentálne hodnotenie vplyvu deliteľa na výsledky konštrukcie.

Translačné kódy sú trieda kódovacích techník používaných na transformáciu ľubovoľných binárnych informačných sekvencií na výstupné sekvencie, ktoré splňajú špecifické štrukturálne požiadavky kanála. Tieto požiadavky vyplývajú z fyzikálnych vlastností systému, ako sú obmedzenia dĺžky behu závislé od média alebo požiadavky na synchronizáciu. Keďže neobmedzené binárne sekvencie tieto požiadavky zvyčajne nespĺňajú, translačné kódy slúžia ako nevyhnutný most medzi logikou reprezentácie informácií a fyzikálnou povahou kanála. Efektívnosť translačného kódu sa meria pomocou kódovej rýchlosti $R = k/n$, kde k je počet informačných bitov a n je dĺžka kódového slova. Kódová rýchlosť je ohraničená teoretickou kapacitou kanála $C = \log_2 \lambda_{\max}$, kde $\lambda_{\max}$ je najväčšie vlastné číslo matice susednosti konečného automatu kanála. Tento vzťah, ktorý pochádza z Shannonovej teórie informácií, poskytuje teoretickú hornú hranicu pre dosiahnuteľnú kódovú rýchlosť pre daný kanál s obmedzeniami.

Dve hlavné konštrukčné techniky študované v tejto práci sú metóda stavovej prechodovej matice a Freiman–Wynerova konštrukcia. Metóda stavovej prechodovej matice odvodzuje maticu B zo stavového diagramu kanála, kde každý prvok b_{ij} udáva počet prechodov zo stavu S_i do stavu S_j . Výpočtom mocnín matice B^n je možné určiť počet prípustných sekvencií dĺžky n a overiť podmienku uskutočniteľnosti $(B^n)_{ii} \geq 2^k$. Freiman–Wynerova konštrukcia ponúka výpočtovo efektívnejší prístup rozdelením každého kódového slova na prefix a sufix, čím sa výrazne znižuje prehľadávaný priestor kandidátnych sekvencií. Kompatibilita prefixu a sufixu je zaručená podmienkou $\text{end}(p) = \text{start}(s)$, ktorá vyžaduje aby terminálny stav prefixu zodpovedal počiatočnému stavu sufixu.

Vlastný nástroj, implementovaný v jazyku C++, predstavuje hlavný praktický príspevok práce. Na rozdiel od predbežných skriptov, ktoré implementovali

iba izolované časti procesu konštrukcie, nástroj správne realizuje celý proces ako jeden súdržný systém organizovaný do šiestich nezávislých modulov. Modul **states** generuje stavový priestor konečného automatu z parametra obmedzenia dĺžky behu r . Pre dané r vzniká $2r$ stavov reprezentujúcich behy núl a jednotiek dĺžky 1 až r . Modul **fsm** implementuje základnú logiku konečného automatu vrátane prechodov medzi stavmi a validácie kódových slov. Modul **capacity** vypočítava teoretickú kapacitu kanála od základu konštrukciou matice susednosti a výpočtom jej najväčšieho vlastného čísla metódou mocninových iterácií. Modul **generator** rekurzívne generuje všetky binárne slová požadovanej dĺžky. Modul **codebook** implementuje úplnú Freiman–Wynerovu konštrukciu prefixov a sufixov s konfigurovateľnými parametrami vrátane deliteľa d . Modul **io_utils** zabezpečuje štruktúrovaný výstup do súborov CSV v priečinkoch s časovým označením. Nástroj prijíma dĺžku kódového slova n , dĺžku informačného bloku k , obmedzenie maximálnej dĺžky behu r a deliteľ d ako nezávislé vstupné parametre a pre každý beh produkuje kompletnú sadu výsledkov vrátane tabuľky prechodov konečného automatu, matice susednosti, výsledkov kapacity, záznamov kódových slov, vybraného kódového slovníka a súhrnných štatistík.

Časová zložitosť celého procesu konštrukcie je dominovaná generovaním záznamov kódových slov s celkovou zložitosťou $\mathcal{O}(n \cdot 2^n)$. Táto exponenciálna závislosť od n je základným obmedzením konštrukcie a priamo vysvetľuje exponenciálny rast výpočtového času pozorovaný v experimentoch. Všetky ostatné komponenty prispievajú najviac polynomiálnou cenou v n alebo r a sú v porovnaní s ňou zanedbateľné.

Štandardná Freiman–Wynerova konštrukcia vždy delí kódové slovo dĺžky n na dve rovnako dlhé časti, čo zodpovedá deliteľu $d = 2$. V tejto práci bol nástroj rozšírený o konfigurovateľný deliteľ d , ktorý umožňuje ľubovoľné rozdelenie kódového slova na prefix a sufix. Dĺžka prefixu sa vypočíta ako $\ell_p = \lceil n/d \rceil$ a dĺžka sufixu ako $\ell_s = n - \ell_p$. Rozšírenie bolo implementované spätne kompatibilným spôsobom, pričom predvolená hodnota $d = 2$ zachováva štandardné správanie. Okrem rozšírenia modulu **codebook** bol aktualizovaný aj modul **io_utils** tak, aby zahrnul deliteľ do názvu výstupného priečinka a do súhrnného súboru CSV. Bol taktiež vyvinutý dávkový spúšťač, ktorý automatizuje vykonávanie veľkého počtu behov konštrukcie naprieč konfigurovateľnými rozsahmi parametrov. Bol taktiež uskutočnený pokus o akceleráciu generovania kódových slov pomocou technológie CUDA na grafickom procesore NVIDIA RTX 4080. Bola vyvinutá úplná imple-

mentácia na GPU nahrádzajúca reťazcovú reprezentáciu stavov konečného automatu celočíselným kódovaním vhodným pre paralelné vykonávanie. Integráciu však nebolo možné dokončiť v rámci rozsahu tejto práce z dôvodu problémov s kompatibilitou medzi nainštalovanou verziou sady nástrojov CUDA a ovládačom grafického procesora.

Experimentálne hodnotenie zahŕňalo 852 behov konštrukcie pre hodnoty deliteľa $d \in \{2, 3, 4, 5, 6, 7\}$, s dĺžkami kódových slov od 10 do 18, dĺžkami informačných blokov od 9 do 17 a obmedzeniami dĺžky behu od 4 do 6. Všetky 852 behov boli úspešne dokončené bez jediného zlyhania. Základné výsledky pre štandardnú konštrukciu s $d = 2$ ukázali, že rýchlostná efektívnosť $\eta = (R/C) \times 100\%$ sa výrazne líši naprieč testovaným parametrickým priestorom, od 27,80% do 99,80%, s celkovým priemerom 63,85%. Dva jasné trendy vyplynuli z výsledkov. Po prvé, rýchlostná efektívnosť klesá so zvyšujúcim sa n pre fixný pomer k/n . Po druhé, efektívnosť sa zvyšuje keď sa k blíži k n , čo potvrdzuje, že prevádzka pri vysokých kódových rýchlostiach je v rámci Freiman–Wynerovej metódy uskutočniteľná a produktívna.

Na účely priameho porovnania deliteľov bolo identifikovaných 48 parametrických kombinácií spoločných pre všetkých šesť dávok. Výsledky sú jednoznačné: naprieč všetkými 48 spoločnými kombináciami bola rýchlostná efektívnosť η identická pre každú testovanú hodnotu deliteľa. Napríklad pri $n = 16$, $k = 15$, $r = 4$ všetkých šesť hodnôt deliteľa produkovalo $\eta = 99,02\%$ s kódovou rýchlosťou $R = 0,9375$ oproti kapacite kanála $C = 0,9468$ bitov na symbol. Výpočtový čas vykazoval rovnako invariantný vzorec, pričom rozdiely medzi hodnotami deliteľa zodpovedali iba bežným výkyvom merania. Najvýznamnejším individuálnym výsledkom bola konštrukcia s takmer maximálnou kapacitou dosiahnutá pri $n = 18$, $k = 17$, $r = 4$, ktorá produkovala rýchlostnú efektívnosť $\eta = 99,75\%$ čo je v rámci 0,0024 bitov na symbol od teoretickej Shannonovej kapacity obmedzeného kanála. Konštrukcia našla 152 848 platných kódových slov v kandidátnom fonde, pričom bolo potrebných iba 131 072 na zostavenie finálneho kódového slovníka 2^{17} kódových slov. Tento výsledok potvrdzuje správnosť a efektívnosť implementácie nástroja a preukazuje, že Freiman–Wynerova konštrukcia je schopná priblížiť sa ku kapacitnej hranici praktických obmedzených kanálov.

Hlavným zistením tejto práce je, že deliteľ d nemá merateľný vplyv na rýchlostnú efektívnosť ani na výpočtový čas Freiman–Wynerovej konštrukcie v testovanom rozsahu parametrov. Konštrukcia je robustná voči variáciám

pomeru rozdelenia blokov a štandardná voľba $d = 2$ nie je ani výhodnejšia ani nevýhodnejšia v porovnaní s alternatívnymi hodnotami. Hlavným praktickým obmedzením zostáva exponenciálny rast výpočtového času s dĺžkou kódového slova, čo robí $n = 18$ praktickou hornou hranicou pre experimenty realizované v tejto práci. Riešenie tohto obmedzenia prostredníctvom algoritmickej optimalizácie alebo úspešnej akcelerácie na GPU by bolo najvýznamnejším smerom pre rozšírenie tejto práce.“

(Claude Sonnet 4.6, 2026)

Bibliography

1. FARKAŠ, P. *Transcontrol Codes*. 1st ed. Bratislava: Wolters Kluwer, 2014. ISBN 978-80-7478-566-5.
2. FREIMAN, C. V.; WYNER, A. D. Optimum block codes for noiseless input restricted channels. *Information and Control*. 1964, **7**(3), 398–415. Available from DOI: 10.1016/S0019-9958(64)90486-3.
3. SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*. 1948, **27**(3), 379–423. Available from DOI: 10.1002/j.1538-7305.1948.tb01338.x.
4. FISHER, R. A.; YATES, F. *Statistical Tables for Biological, Agricultural and Medical Research*. London: Oliver and Boyd, 1938.
5. NVIDIA CORPORATION. *CUDA C++ Programming Guide* [online]. 2026. Version 13.2 [visited on 2026-05-01]. Available from: <https://docs.nvidia.com/cuda/cuda-programming-guide/>.
6. *Opatrenie číslo 1/2024–O. Používanie umelej inteligencie na Slovenskej technickej univerzite v Bratislave* [online]. 2024-11-27. [visited on 2024-12-31]. Available from: https://www.stuba.sk/buxus/docs/stu/pracoviska/rektorat/odd_pravne_organizacne/28.11.2024%20opatrenie_AI-signed.pdf.

Usage of artificial intelligence tools

[6]

Anthropic (2026), Claude Sonnet 4.6, Abstrakt, preklad abstraktu do slovenčiny.

Anthropic (2026), Claude Sonnet 4.6, kapitola 1, konzultácia teoretických základov translačných kódov.

Anthropic (2026), Claude Sonnet 4.6, kapitola 3, konzultácia návrhu a implementácie modulárneho nástroja v jazyku C++.

Anthropic (2026), Claude Sonnet 4.6, kapitola 4, konzultácia štruktúry experimentálneho hodnotenia a analýzy výsledkov.

Anthropic (2026), Claude Sonnet 4.6, dodatok 2, generovanie slovníka kľúčových pojmov.

Appendix A: Algorithms

Algorithm 1 Generate all binary words of length ℓ

Require: $\ell \geq 1$

Ensure: All binary strings of length ℓ

$result \leftarrow \emptyset$

call RECURSE(ℓ , “”, $result$)

return $result$

procedure RECURSE(ℓ , $current$, $result$)

if $|current| = \ell$ **then**

$result \leftarrow result \cup \{current\}$

return

end if

call RECURSE(ℓ , $current + \text{“0”}$, $result$)

call RECURSE(ℓ , $current + \text{“1”}$, $result$)

Algorithm 2 Validate run-length constraint for word w

Require: Binary word w of length n , maximum run length r

Ensure: **true** if w satisfies the run-length constraint, **false** otherwise

```
  if  $|w| = 0$  then
    return true
  end if
   $runLength \leftarrow 1$ 
  for  $i \leftarrow 2$  to  $|w|$  do
    if  $w[i] = w[i - 1]$  then
       $runLength \leftarrow runLength + 1$ 
      if  $runLength > r$  then
        return false
      end if
    else
       $runLength \leftarrow 1$ 
    end if
  end for
  return true
```

Algorithm 3 Compute largest eigenvalue via power iteration

Require: Adjacency matrix $\mathbf{A}$ of size $2r \times 2r$, maximum iterations $maxIter$, tolerance

ε

Ensure: Largest eigenvalue $\lambda_{\max}$

$\mathbf{x} \leftarrow \mathbf{1}$ {initialise with ones vector}

$\lambda_{old} \leftarrow 0$

for $iter \leftarrow 1$ **to** $maxIter$ **do**

$\mathbf{y} \leftarrow \mathbf{A} \cdot \mathbf{x}$

$norm \leftarrow \|\mathbf{y}\|_2$

if $norm = 0$ **then**

return 0

end if

$\mathbf{x} \leftarrow \mathbf{y}/norm$

$\lambda_{new} \leftarrow norm$

if $|\lambda_{new} - \lambda_{old}| < \varepsilon$ **then**

break

end if

$\lambda_{old} \leftarrow \lambda_{new}$

end for

return λ_{new}

Algorithm 4 Generate codeword records via Freiman–Wyner construction

Require: Codeword length n , run-length constraint r , division factor d **Ensure:** Set of valid **CodewordRecord** structures

```
 $\ell_p \leftarrow \lceil n/d \rceil$ 
 $\ell_s \leftarrow n - \ell_p$ 
 $P \leftarrow \text{GENERATEBINARYWORDS}(\ell_p)$ 
 $S \leftarrow \text{GENERATEBINARYWORDS}(\ell_s)$ 
 $records \leftarrow \emptyset$ 
for each  $p \in P$  do
   $s_1 \leftarrow \text{APPLYFIRSTBLOCK}(p, r)$ 
  if  $s_1 = \text{INVALID}$  then
    continue
  end if
  for each  $q \in S$  do
     $s_2 \leftarrow \text{APPLYBLOCKFROMSTATE}(s_1, q, r)$ 
    if  $s_2 = \text{INVALID}$  then
      continue
    end if
     $codeword \leftarrow p \parallel q$ 
    if  $|codeword| \neq n$  or not  $\text{ISVALIDWORD}(codeword, r)$  then
      continue
    end if
    if  $codeword \notin records$  then
       $records \leftarrow records \cup \{(p, s_1, q, s_2, codeword)\}$ 
    end if
  end for
end for
return  $records$ 
```

Algorithm 5 Select codebook of size 2^k from valid codeword records

Require: Codeword length n , information block length k , run-length constraint r ,
division factor d

Ensure: Codebook of exactly 2^k codewords

$required \leftarrow 2^k$

$records \leftarrow \text{GENERATECODEWORDRECORDS}(n, r, d)$

if $|records| < required$ **then**

return FAILURE

end if

SHUFFLE($records$) {Fisher–Yates shuffle}

$records \leftarrow records[1 \dots required]$

return $records$

Appendix B: Dictionary

Admissible sequence – A binary sequence that satisfies all structural constraints imposed by the target channel, such as run-length limitations or forbidden patterns.

Adjacency matrix – A square matrix $\mathbf{A}$ derived from the finite state machine of a constrained channel, where entry A_{ij} counts the number of valid transitions from state i to state j . Used to compute channel capacity via eigenvalue analysis.

Channel capacity – The theoretical maximum rate at which information can be reliably transmitted over a constrained channel, given by $C = \log_2 \lambda_{\max}$ bits per symbol, where $\lambda_{\max}$ is the largest eigenvalue of the adjacency matrix.

Code rate – The ratio $R = k/n$, expressing how many information bits are transmitted per channel symbol. A higher code rate indicates greater efficiency.

Codebook – A complete mapping from all 2^k possible input messages to their corresponding channel-admissible codewords of length n .

Codeword – A binary sequence of length n that satisfies the channel constraint and is used to represent a k -bit input message during transmission.

Constrained channel – A communication or storage channel that prohibits certain binary patterns in transmitted sequences, typically due to physical characteristics such as timing recovery requirements or energy dissipation limitations.

Division factor – A parameter $d \geq 2$ that controls how a codeword of length n is split into a prefix and a suffix during the Freiman–Wyner construction. The standard value is $d = 2$, producing two equal or near-equal blocks.

Finite state machine (FSM) – A mathematical model used to describe the set of admissible sequences for a constrained channel. Each state represents a relevant symbol history, and transitions between states are determined by the channel constraint.

Freiman–Wyner construction – A technique for constructing translation code codebooks by dividing each codeword into a prefix and a suffix, generating valid sequences for each part independently, and combining compatible pairs to form complete codewords.

Largest eigenvalue – The dominant eigenvalue $\lambda_{\max}$ of the FSM adjacency matrix, computed via power iteration. Determines the channel capacity through the relation $C = \log_2 \lambda_{\max}$.

Power iteration – An iterative numerical method for computing the largest eigenvalue of a matrix by repeatedly multiplying a vector by the matrix and normalising the result until convergence.

Prefix – The first block of a codeword in the Freiman–Wyner construction, of length $\lceil n/d \rceil$. Its terminal FSM state must be compatible with the starting state of the corresponding suffix.

Rate efficiency – A metric $\eta = (R/C) \times 100\%$ expressing the achieved code rate as a percentage of the theoretical channel capacity. A value of 100% indicates operation at the Shannon limit.

Run-length constraint – A restriction on the maximum number of consecutive identical symbols permitted in a sequence. Parameterised by r in this thesis, where no run of zeros or ones may exceed r symbols in length.

Shannon limit – The theoretical upper bound on achievable code rate for a given channel, equal to the channel capacity C . A construction operating at the Shannon limit transmits information as efficiently as the channel permits.

State space – The complete set of FSM states for a given run-length constraint r , consisting of $2r$ states representing runs of zeros and ones of lengths 1 through r .

Suffix – The second block of a codeword in the Freiman–Wyner construction, of length $n - \lceil n/d \rceil$. Its starting FSM state must match the terminal state of the corresponding prefix.

Translation code – A class of constrained codes that map unconstrained k -bit input blocks to constrained n -bit output codewords, ensuring compatibility between information sequences and channel requirements.